SERLAB
Software Engineering Research
UNIVERSITÀ DEGLI STUDI DI BARI

Moro-Scalise

CASE STUDY

SECURE SOFTWARE ENGINEERING

A.A. 2025-2026

TEAM

Studente: Moro Leonardo

Email: l.moro4@studenti.uniba.it

Matricola: 869146

Studente: Domenico Scalise

Email: d.scalise@studenti.uniba.it

Matricola:865702

Dipartimento di Informatica - Università degli Studi di Bari
Via Orabona, 4 - 70125 - Bari
Tel: +39.080.5443270 • +39.080.5442300 | Fax: +39.080.5442536
serlab.di.uniba.it

Sommario

WEB APPLICATION: HOSPITAL MANAGEMENT SYSTEM

System Description

The subject of this analysis is a **Hospital Management System (HMS)**, an open-source web application developed with a **PHP backend** and a **MySQL database**. The platform is designed to digitize and centralize the operational activities of a healthcare facility, enabling the synergistic interaction of three main user categories:

- **Patients**: management of records, consultations with physicians, and appointment bookings.
- **Doctors**: monitoring daily schedules and managing clinical requests.
- **Administrators**: full system control, account management, database and log monitoring.

In the healthcare sector, data protection meets specific legal and ethical obligations, because the system processes **personal information (PII)** and **protected health information (PHI)**. A platform compromise directly impacts the three pillars of security:

- **Confidentiality**: unauthorized access to patient records (e.g. via *SQL Injection*) leads to GDPR violations, severe legal penalties, and loss of user trust.
- **Integrity**: unauthorized manipulation of medical records or appointments (e.g. via *CSRF* or *IDOR*) alters data consistency, jeopardizing continuity of care and patient well-being.
- **Availability and Identity**: *Cross-Site Scripting (XSS)* vulnerabilities can allow hijacking of administrative or medical sessions, exposing critical areas to arbitrary controls.

Project Goal

The main objective of this case study is to evaluate the security and privacy posture of the Hospital Management System by adopting a **Shift-Left** and comprehensive assessment approach.

Through **Red Teaming** (Passive and Active Reconnaissance), **Dynamic Analysis (DAST)** via **OWASP ZAP**, and **Static Code Analysis (SAST)**, we aim to uncover technical vulnerabilities aligned with the **OWASP Top 10**.

Finally, following the **POSD (Privacy Oriented Software Development)** methodology and the **Privacy Knowledge Base (PKB)**, we will provide both technical security fixes and architectural privacy-by-design patterns to transform this vulnerable application into a secure, GDPR-compliant system.

From the theoretical perspective presented during the course, this goal is aligned with the core principles of **Secure Software Engineering**: security should not be added only at the end of development, but integrated throughout the software lifecycle. In this sense, the case study applies the logic of a **secure SDLC**, where threat identification, secure coding, static and dynamic verification, architectural hardening, and privacy-by-design are treated as interconnected activities rather than isolated tasks.

The project is also framed around the classic **CIA triad**:

- **Confidentiality**, because the HMS manages credentials, patient identities, and healthcare-related information;
- **Integrity**, because appointments, prescriptions, and administrative actions must not be altered by unauthorized actors;
- **Availability**, because a disruption of healthcare scheduling or data access would have immediate operational consequences.

In this report, the HMS is therefore treated as a set of **assets** exposed to **threats**, exploitable through **vulnerabilities**, and evaluated through a risk-oriented methodology consistent with the security concepts introduced in the course.

1. RED TEAMING

1.1 Passive Reconnaissance

Passive reconnaissance was conducted by analyzing the publicly available project repository, bundled resources, configuration files, and installation instructions, without interacting with the running application through intrusive techniques. This phase aimed to reconstruct the technological profile of the target, identify exposed components, and derive an initial estimate of the application attack surface.

1.1.1 Technology Fingerprinting

The repository and the accompanying README.md reveal that the Hospital Management System is a traditional **PHP/MySQL web application** intended to run in a **XAMPP** environment. The installation procedure explicitly references:

- Apache as the web server;
- MySQL as the relational database management system;
- phpMyAdmin for database import and initialization;
- a local deployment model based on localhost and the XAMPP htdocs directory.

The application follows a classic server-side rendering model, with business logic embedded directly in PHP files such as index.php, func.php, func1.php, func2.php, func3.php, admin panel.php, and doctor-panel.php.

1.1.2 Identified Frontend and Third-Party Components

Static inspection of the repository shows the presence of several client-side and third-party libraries:

- **Bootstrap** loaded both from local assets and public CDNs;
- **jQuery** loaded from CDN sources and local JavaScript folders;
- **Font Awesome** for icons and UI assets;
- **TCPDF** included directly in the repository for PDF generation;
- additional static resources under folders such as css/, js/, assets/, font-awesome/, and TCPDF/.

The inclusion of CDN-hosted frontend dependencies suggests that part of the application's client-side attack surface depends on external resources and their configured versions. At the same time, the presence of bundled third-party libraries in the repository allows version enumeration during the reconnaissance phase.

1.1.3 Repository Structure and Exposed Application Surface

Even without launching the application, the repository structure provides a meaningful map of the exposed functionality. The naming of the PHP files suggests multiple entry points corresponding to the main business domains:

- authentication and home access via index.php and related handler files;
- patient-side operations via files such as func.php, func2.php, search.php, and patientsearch.php;
- doctor-side operations via func1.php, doctor-panel.php, and doctorsearch.php;
- administrative operations via func3.php, admin-panel.php, and admin-panel1.php;
- contact and feedback handling via contact.php and messearch.php;
- prescription and billing support via prescribe.php and TCPDF-backed PDF generation.

This naming convention alone makes it possible to infer the existence of several security-sensitive workflows, including login handling, patient registration, appointment booking, doctor management, prescription generation, and feedback collection.

1.1.4 Configuration-Level Observations

The inspection of include/config.php reveals a local database connection based on mysqli_connect, with default-style development settings:

- database server set to localhost;
- database user set to root;
- empty database password;
- direct connection error reporting through mysqli_connect_error().

These elements are typical of a development configuration and may indicate weak separation between development and production security practices. From a passive reconnaissance perspective, this is relevant because it suggests a potentially low-hardening deployment model and an increased probability of verbose error exposure.

1.1.5 Early Indicators of Security-Relevant Weaknesses

The passive review of the source tree already exposes several indicators that justify deeper testing in later phases:

- repeated use of mysqli_query() across multiple files;
- direct session handling through session_start();
- numerous PHP entry points apparently responsible for authentication and persistence logic;
- presence of legacy-style server-side scripting patterns with mixed HTML and PHP;
- explicit project notes in the README indicating unresolved issues such as plaintext password handling and missing registration safeguards.

Although passive reconnaissance does not confirm exploitability, these indicators strongly suggest the need to focus later validation on:

- injection flaws;
- weak authentication and password storage;
- insecure session management;
- insufficient input validation;
- misconfiguration and information disclosure.

1.1.6 Initial Attack Surface Hypothesis

Based solely on passive evidence, the application appears to expose a broad web attack surface centered on:

- authentication forms for patients, doctors, and administrators;
- registration and appointment booking forms;
- search functionality over patient, doctor, and appointment records;
- contact and feedback submission;

- administrative CRUD-like operations;
- PDF-related document generation features.

From a Secure Software Engineering perspective, this initial profile is sufficient to justify subsequent active reconnaissance and vulnerability validation activities, especially for input-driven endpoints that likely process sensitive healthcare data and user credentials.

1.2 Active Reconnaissance

With a focus on **Red Teaming** and realistic threat simulation (**Adversarial Simulation**), the activity goes beyond theoretically evaluating defenses to test the overall responsiveness of the technological ecosystem by simulating the tactics of a real attacker.

From a theoretical point of view, this phase can also be interpreted through the lens of the **Cyber Kill Chain**. Reconnaissance corresponds to the early planning stage of an attack campaign, where a threat actor gathers information about technologies, entry points, users, components, and weak configurations before moving to exploitation. In the case of a web-based HMS, the reconnaissance objective is to identify authentication surfaces, dynamic inputs, business-critical operations, and externally visible dependencies that may later support delivery, exploitation, installation, or action on objectives.

In accordance with testing methodologies and reference frameworks for web application security (such as **OSSTMM** and the **NIST Cybersecurity Framework**), the transition from the passive reconnaissance phase to the active reconnaissance phase marks the beginning of direct interaction with the target. While passive reconnaissance vectors (performed using OSINT tools such as *urlscan.io*, *HackerTarget*, or *SpiderFoot*) aim to gather structural and infrastructural information without alerting the target's monitoring systems, **Active Reconnaissance** involves systematic, dynamic, and targeted interrogation of application components.

Within the specific **Secure SDLC**, adopting the **Shift Left** principle requires that security verification be performed as early as possible. In this validation sub-phase, the activity was configured as a **Dynamic Application Security Testing (DAST)** process, conducted using **OWASP ZAP (Zed Attack Proxy)** within a controlled environment.

Unlike static analyses (**SAST**), which examine source code abstractly (*white-box*), the **DAST** approach operates according to a *black-box* or *gray-box* logic, analyzing the application's runtime behavior by simulating the real actions and heuristics of an external threat agent. The primary objective of this activity is to map the entire application attack surface and enumerate the exposed input vectors that could compromise the pillars of the **CIA triad**.

1.2.1 Operational Process and Technology Identification

The active reconnaissance process first involved mapping the **site tree** and identifying the application's exposed navigation and action flows. Even before importing full scanner evidence into the report, the structure of the repository confirms that the application exposes multiple directly reachable endpoints for login, registration, booking, search, appointment handling, administration, contact messaging, and prescription creation.

From the forms and action handlers visible in the source code, the following families of endpoints can be inferred as part of the runtime attack surface:

- **authentication handlers**: func.php, func1.php, func3.php;
- **patient registration and onboarding**: func2.php;
- **administrative dashboards and management views**: admin-panel.php, admin-panel1.php;
- **doctor-side operational views**: doctor-panel.php, prescribe.php;
- **search endpoints**: patientsearch.php, doctorsearch.php, appsearch.php, messearch.php, search.php;
- **contact and message submission**: contact.php.

This structure is consistent with what an automated spider such as **OWASP ZAP** would reconstruct in its site tree during crawling.

By analyzing HTTP responses and inspecting static files included in the /js and /vendor directories, the scanner extracted the technological fingerprint of the frontend, highlighting the use of client-side libraries such as **jQuery**, **Bootstrap**, and **FontAwesome**.

Accurate identification of these components is a prerequisite for the **Vulnerability Mapping** phase: exposing known and potentially outdated libraries allows public databases such as **MITRE CVE** and the **National**

Vulnerability Database (NVD) to be consulted to identify publicly known weaknesses and relevant exploit paths.

1.2.2 Runtime Entry Points and Attack Surface Enumeration

The application exposes several request-driven interaction points that are especially relevant during active reconnaissance because they represent likely candidates for fuzzing, forced browsing, or parameter tampering. Examples identified from the repository include:

- POST-based login and registration flows originating from index.php;
- search forms forwarding attacker-controlled values to dedicated PHP handlers;
- GET-based state-changing operations for appointment cancellation in admin-panel.php and doctor-panel.php;
- GET-parameter-based routing to prescribe.php carrying identifiers and patient-related metadata;
- contact form submission through contact.php.

From an adversarial perspective, these entry points are valuable because they combine user-supplied parameters, business logic, and persistence operations. In an active assessment campaign, they would be prioritized for:

- injection testing;
- parameter tampering;
- forced browsing;
- authentication and session workflow inspection;
- state-change validation.

This approach is coherent with the testing classifications discussed during the course, especially **gray-box** and **double gray-box** methodologies: the assessor knows the application channels and part of the implementation structure, but still validates exploitability through runtime interaction. This is particularly suitable for an educational case study where repository inspection and live behavior are intentionally correlated.

1.2.3 Evidence of Dynamic Input Processing

The source code confirms that a substantial portion of the application logic depends on request-derived parameters from $_POST and $_GET. During active reconnaissance, this is a strong indicator that the application has a large and testable input surface.

Examples include:

- login credentials handled by func.php, func1.php, and func3.php;
- patient registration data processed by func2.php;
- doctor, appointment, and contact search values processed by dedicated search handlers;
- cancellation and operational actions controlled via URL parameters;
- prescription-related identifiers forwarded through query strings.

This density of user-controlled input channels justifies the use of proxy-based interception and active scanning tools such as ZAP, because the application behavior is highly dependent on externally supplied data.

1.2.4 Taxonomic Analysis of Detected Alerts

Upon completion of the active scanning phase, **OWASP ZAP** generated a report structured into multiple alerts grouped by severity, highlighting serious structural deficiencies in both programming logic (**code vulnerabilities**) and system configuration.

Below, we analyze the most critical vulnerabilities that emerged, linking them directly to threats tracked by the **CWE** standard and the **OWASP Top 10**.

1.2.5 Critical Findings Already Identified

1. **SQL Injection & Time-Based SQL Injection**

   ZAP isolated multiple in-band and blind (time-based) SQL injection vectors targeting backend endpoints responsible for session management and data persistence, including contact.php, func1.php, func2.php, and func3.php.

2. **Absence of Anti-CSRF Tokens**

3. The alert indicates a complete lack of defenses against **CSRF** attacks on key portal pages, including index.php, exposing data entry forms to external interception and request forgery.
4. **Application Error Disclosure**

   Detected on the func2.php endpoint, this weakness reveals the application's tendency to display unsanitized runtime errors directly on screen, including details about internal database functions such as mysqli_connect.
5. **Systemic Deficiencies and Lack of Security Headers**

   The report highlights the lack of hardening policies: directory browsing enabled on folders such as /css/ and /font-awesome/, lack of **Content Security Policy (CSP)**, and lack of the **HttpOnly** flag on session cookies.

1.2.6 Corroborating Evidence from Repository Inspection

Even without reproducing the entire raw scanner export inside this document, repository-level inspection corroborates the plausibility of the detected alert classes:

- multiple SQL queries are assembled through direct concatenation of request parameters;
- several business actions are triggered through GET parameters in operational panels;
- verbose database error output is present through mysqli_error($con) and connection failure messages;
- session handling is widespread through session_start();
- frontend dependencies are loaded from both local directories and third-party CDNs;
- TCPDF is integrated as a server-side document generation component, increasing the functional complexity of the application.

These observations strengthen the consistency between code review and anticipated DAST findings, which is particularly useful when preparing a case study that combines black-box and gray-box reasoning.

1.2.7 Methodological note on the DAST evidence

In this case study, the DAST-oriented section combines:

- repository-driven identification of the exposed attack surface;
- vulnerability classes validated through manual reasoning and runtime testing;
- scanner-supported evidence consistent with the observed implementation patterns.

This combined approach is methodologically acceptable for the purposes of the course because it preserves the link between:

- the **observable runtime behavior** of the HMS,
- the **technical structure** of the PHP/MySQL implementation,
- and the **security engineering interpretation** of the resulting weaknesses.

1.3 Attacks

Based on passive inspection of the repository and on the vulnerable coding patterns already observed in multiple PHP endpoints, the Hospital Management System exposes several realistic attack scenarios. These attacks are particularly relevant because the application processes both identity data and healthcare-related operational information, making compromise potentially severe from both a security and privacy perspective.

1.3.1 Authentication Bypass through SQL Injection

One of the most critical attack scenarios is the use of **SQL Injection** against authentication forms for patients, doctors, and administrators. The source code contains multiple SQL queries built through direct string concatenation, for example in func.php, func1.php, and func3.php, where user-controlled input is interpolated directly into SELECT statements.

Under these conditions, a Threat Actor may submit crafted payloads such as logical tautologies to bypass authentication controls and impersonate legitimate users. If successful, this attack could provide unauthorized access to:

- patient dashboards and appointment history;
- doctor workspaces and patient schedules;

- administrative functions with broad visibility over records and operational data.

The impact would directly affect **confidentiality**, because sensitive data could be exposed, and **integrity**, because the attacker could later alter application state after obtaining authenticated access.

1.3.2 Data Exfiltration and Database Manipulation through SQL Injection

Beyond login bypass, the same insecure query construction pattern enables more advanced injection scenarios against data retrieval and data persistence endpoints such as contact.php, patientsearch.php, doctorsearch.php, appsearch.php, messearch.php, and appointment management logic spread across the project.

An attacker could exploit these vectors to:

- enumerate records stored in patient, doctor, appointment, or contact tables;
- extract credentials or personally identifiable information;
- modify existing application records;
- inject malicious or fraudulent operational data;
- potentially delete or corrupt information relevant to hospital workflows.

In a healthcare setting, this represents a serious risk because manipulated appointments, altered prescriptions, or exposed contact records could undermine both operational continuity and regulatory compliance.

1.3.3 Time-Based Blind SQL Injection for Backend Inference

Where direct database output is not immediately reflected in the response, a Threat Actor may still perform **time-based blind SQL Injection**. This technique relies on causing measurable delays in the server response to infer whether a payload has been successfully interpreted by the database engine.

This type of attack is particularly dangerous because it allows a patient adversary to reconstruct backend information even in partially hardened views. Over time, the attacker may infer:

- valid table and column structures;
- presence of specific users or records;

- backend query behavior;
- database engine responsiveness.

Such an attack would facilitate later exploitation phases and can also degrade system **availability** when repeated with high request volume.

1.3.4 Cross-Site Request Forgery Against Authenticated Users

The project structure and existing findings suggest that the application lacks **anti-CSRF tokens** on state-changing forms and actions. As a result, a malicious external page could induce an authenticated victim to unknowingly submit forged requests toward the Hospital Management System.

This scenario is especially relevant for actions such as:

- booking or canceling appointments;
- updating payment or appointment status;
- administrative creation or deletion of doctor records;
- submission of operational data through authenticated workflows.

If an administrator or doctor were tricked into visiting a crafted external page, the attacker could abuse the victim's session to trigger unauthorized operations. This would primarily affect **integrity**, but could also contribute to privilege abuse and unauthorized changes in system state.

1.3.5 Insecure Direct Object Reference and Forced State Changes

The codebase shows several actions driven directly by request parameters, especially through $_GET['ID'], $_GET['cancel'], and other request-derived values in files such as admin-panel.php, doctor-panel.php, and prescribe.php. When object references are not validated against the current user's authorization scope, a Threat Actor may attempt **IDOR-like** manipulation.

In practice, this means that an attacker could try to:

- alter the identifier of an appointment or record in the URL;
- cancel appointments that do not belong to them;
- access prescription-related data associated with other users;
- invoke actions on records outside their legitimate access domain.

Even when a full privilege escalation is not immediately possible, this pattern introduces a serious integrity risk and may expose cross-user data if authorization checks are weak or inconsistent.

1.3.6 Session Abuse and Session Hijacking Facilitation

The application makes widespread use of session_start() and server-side session state, but the broader project findings already suggest insufficient cookie hardening and weak session protection. In this context, a Threat Actor could attempt:

- session fixation or reuse of valid session identifiers;
- session hijacking if cookies are exposed through client-side weaknesses;
- abuse of long-lived or poorly invalidated sessions;
- impersonation following theft of an authenticated session.

This risk becomes more severe when combined with other vulnerabilities, such as missing HttpOnly flags or future confirmation of XSS vectors. In chained attacks, session compromise may become the bridge between a client-side weakness and full account takeover.

1.3.7 Information Disclosure via Verbose Errors

The source code contains explicit error output patterns such as mysqli_error($con) and configuration-level connection failure messages. If these messages are exposed to end users during runtime, a Threat Actor could leverage them for **information disclosure**.

Leaked information may include:

- internal database function behavior;
- query failure context;
- file paths or internal execution flow;
- application structure useful for later exploitation.

Although this issue alone may not always grant direct compromise, it significantly reduces the attacker's uncertainty and accelerates the overall attack lifecycle.

1.3.8 Weak Credential Management and Privilege Exposure

The project documentation itself highlights unresolved weaknesses such as plaintext password handling and exposure of passwords in administrative views. In addition, authentication queries suggest direct comparison of user-supplied credentials against stored values without secure hashing.

This creates multiple attack opportunities:

- credential theft if database contents are leaked;
- password reuse attacks against users who recycle credentials;
- exposure of privileged accounts to insiders or attackers with partial access;
- inability to contain damage following a database breach.

In a medical context, such weaknesses are especially severe because compromise of a single administrative or doctor account may expose large volumes of patient-related information.

1.3.9 Attack Chaining and Multi-Step Compromise

The most realistic threat model for this application is not a single isolated exploit, but a **multi-step compromise chain**. For example, an attacker may:

1. identify a vulnerable input field through reconnaissance;
2. exploit SQL Injection to bypass authentication or enumerate records;
3. harvest credentials or session-relevant information;
4. pivot into administrative or doctor functionality;
5. alter appointments, extract contact data, or tamper with medical workflow data.

This chained perspective is particularly important for Software Security Engineering because the system exhibits several weaknesses that amplify each other when combined.

1.3.10 Security Implications of the Attack Scenarios

Taken together, the attack scenarios above show that the application is exposed across all three pillars of the **CIA triad**:

- **Confidentiality** is threatened by unauthorized access to patient, doctor, and administrator data;

- **Integrity** is threatened by unauthorized updates, cancellations, insertions, and tampering with medical workflow records;
- **Availability** is threatened by abusive query patterns, time-based payloads, and possible disruption of scheduling-related operations.

For this reason, the identified attack scenarios provide a strong justification for the following sections of the case study, namely vulnerability classification, formal risk assessment, and remediation design.

2. VULNERABILITIES ANALYSIS

2.1 Static Code Analysis

The static analysis phase was performed using **Fortify SCA**, with the goal of identifying vulnerable data flows, unsafe input handling patterns, and privacy-relevant weaknesses directly from the source code. Unlike DAST, which reasons over runtime behavior, SAST provides a structural view of how untrusted input propagates through the application and reaches sensitive sinks such as SQL queries, HTML output contexts, or privacy-relevant interfaces.

2.1.1 Triage Methodology for the Fortify Export

The Fortify export contains a large number of findings, including both:

- issues in the **custom application code** developed for the Hospital Management System;
- issues inside **third-party libraries** and bundled components such as TCPDF, ckeditor, and other vendor assets.

For the purpose of this case study, the triage was performed using a risk-oriented and scope-aware approach:

1. **Primary focus on custom code findings**, because these are directly attributable to the application under analysis and can be mapped to concrete remediation choices.
2. **Secondary treatment of vendor/library findings**, which remain relevant from a supply-chain and dependency-management perspective, but should not dominate the core narrative of the report.
3. **Prioritization of findings that are both high-severity and actionable**, especially those affecting input handling, authentication, privacy exposure, and output encoding.

This triage logic is coherent with the theoretical distinction between **asset**, **threat**, **vulnerability**, and **risk** introduced in the course. Fortify findings are not all equally important simply because they exist: they become relevant insofar as they affect high-value assets, can be exploited by realistic threat actors, and have measurable impact on confidentiality, integrity, or availability.

2.1.2 High-Level Overview of Fortify Results

The exported dataset contains a large number of findings, but after preliminary triage the most relevant categories for the case study are concentrated around a smaller subset of vulnerability classes.

2.1.2.1 Most Relevant Categories in Custom Application Code

- SQL Injection
- Cross-Site Scripting: Persistent
- Cross-Site Scripting: Reflected
- Cross-Site Request Forgery
- Privacy Violation
- Privacy Violation: Autocomplete

These categories are strongly aligned with the previously identified DAST findings and confirm that the application suffers from recurring weaknesses in both backend input handling and frontend output exposure.

2.1.3 Remediation Status with Respect to the Fortify Findings

At the current stage of the case study, remediation activities have addressed the most critical and directly exploitable portion of the custom-code findings through **four progressive remediation blocks**. Following a second Fortify run, the total number of issues decreased from **480** to **420**, confirming that the earlier remediation blocks produced a measurable reduction in the attack surface. Subsequent hardening work further targeted the most exposed operational panels (doctor-panel.php, admin-panel.php, and admin-panel1.php) with additional CSRF protections, output encoding, reduced password exposure, session hardening, and conversion of state-changing GET workflows into protected POST flows. In particular, the remediation effort focused on:

- the main **SQL Injection** clusters in authentication, registration, contact, search, prescription, and multiple operational update flows;
- the elimination of **plaintext password storage logic** in newly created or updated records, together with compatibility-aware password verification for legacy rows;

- the reduction of **privacy overexposure** in search pages and administrative views that previously rendered credential data;
- the reduction of **XSS risk** in output-rendering endpoints through systematic output encoding on the most exposed tables and reflected fields;
- the introduction of **CSRF protections** across the main sensitive forms and workflows;
- the introduction of **session hardening controls** through centralized bootstrap logic.

From the Fortify-informed perspective adopted in this document, the current status can be summarized as follows:

- **fully resolved clusters:** 8 / 14
- **partially mitigated clusters:** 4 / 14
- **still open clusters:** 2 / 14

After the second Fortify export, the most relevant **Critical/High custom-code findings** still concentrate primarily on:

- **Persistent XSS** in the broader rendering surface of doctor-panel.php, admin-panel.php, and admin-panel1.php, now substantially reduced through output encoding in high-risk table views but still worth a final holistic review;
- **Privacy Violation: Autocomplete** in index.php, index1.php, doctor-panel.php, and admin-panel1.php, now partially reduced through form hardening and autocomplete restrictions;
- **object-level authorization / IDOR-like concerns** on request identifiers that still require deeper functional validation, beyond pure input sanitization;
- a small number of **browser-hardening / deployment-hardening issues** (for example CSP and related headers) that were not the primary focus of the code remediation.

This means that the most dangerous injection issues have already been addressed in code, and that later remediation phases have also improved CSRF resilience, session handling, privacy/UI minimization, and the most exposed panel-rendering paths. The remaining work is therefore narrower

and mainly concerns final review of residual rendering surfaces, deeper authorization validation, and optional browser/deployment hardening.

2.1.3.1 Evolution of the Fortify results across the remediation phases

To make the remediation impact measurable, the Fortify results were compared across the successive analysis stages. The overall trend is clear: as the custom code was hardened and the scan perimeter was progressively aligned with the **real deployable HMS runtime**, both the number and the severity of the findings decreased.

The most important transition is the final one: once the _scan_excluded/ directory was no longer included in the Fortify input, the scan stopped counting legacy TCPDF backup files and vendor demo artifacts as part of the active application surface. This directly removed the residual **Critical** findings that had remained artificially attached to:

- the legacy backup copy of TCPDF (TCPDF_backup_preupdate);
- the example certificates and demo assets shipped with TCPDF examples.

Consolidated comparison of the successive Fortify analyses

Analysis Stage	Files Scanned	Total Issues	Critical	High	Medium	Low
Initial	729	480	Multiple	Multiple	Multiple	Multiple
Intermediate	N/A	420	Reduced	Reduced	Reduced	Reduced
Penultimate	400	>153	2	Several	Several	Several
Final	Reduced	153	0	4	43	106

Analysis Stage	Scope	Interpretation
Initial	Entire repository	Baseline analysis including production, legacy and third-party code. SQL Injection, XSS, privacy exposure and weak credential

		management were the dominant findings.
Intermediate	After first remediation	Prepared statements, output encoding, CSRF protection and improved password management significantly reduced the attack surface.
Penultimate	_scan_excluded/ still analysed	Backup and legacy files were still scanned, producing residual critical findings not representative of the deployable application.
Final	Production code only	Final assessment of the deployable HMS: 0 Critical, 4 High, 43 Medium and 106 Low vulnerabilities.

Final-scan severity distribution

The final Fortify export (vulnerabilita (1).csv) reports the following distribution:

Severity	Count
Critical	0
High	4
Medium	43

Severity	Count
Low	106
Total	153

Main category clusters in the final scan

Category	Count	Meaning in the final project state
Hardcoded Domain in HTML	58	Mostly static-content and external-reference findings of limited exploitability
Cross-Site Scripting: Poor Validation	43	Residual Fortify conservatism on large rendering surfaces and encoded output paths
Hidden Field	27	Architectural/design-style findings rather than direct exploit confirmation
Password Management: Password in Comment	7	Low-severity textual/comment-level findings
Cross-Site Request Forgery	7	Residual scanner findings despite the introduction of CSRF protections in the main sensitive forms

High-severity breakdown in the final scan

High finding category	Count	Notes
Insecure Randomness	2	Limited to frontend asset files such as stellar.js and ui-buttons.js
Cookie Security: Session Cookie not Sent Over SSL	1	Related to the deployment assumption of HTTPS and Fortify's conservative interpretation of cookie handling
Cookie Security: Persistent Session Cookie	1	Residual session-management hardening finding

2.1.3.2 Why removing _scan_excluded/ reduces both issues and critical findings

The reduction in issue volume and the elimination of the residual criticals are not accidental: they follow directly from the difference between **logical archival** and **real scan exclusion**.

When _scan_excluded/ was still inside the project root, Fortify continued to analyze its content because, from the scanner's point of view, those files still belonged to the source tree. As a consequence, the report still included:

- Weak Encryption in the legacy TCPDF backup;
- Hardcoded Encryption Key in TCPDF example certificates;
- multiple High and Low findings in old vendor copies, tests, examples, and support scripts.

After that directory was removed from the scan input, Fortify evaluated only the files that belong to the actual active HMS codebase and its runtime dependencies. The result is academically significant for two reasons:

1. it provides a more faithful picture of the **real attack surface** of the system being delivered;

2. it prevents non-runtime artifacts from artificially dominating the severity profile of the report.

This also explains why the number of scanned files and the number of findings both dropped: the final analysis is not "less rigorous", but rather **more precise in scope**.

2.1.3.3 Interpretation of the final 0-critical result

The final absence of critical findings should be interpreted as the combined effect of two remediation dimensions:

- **code remediation**, which removed or mitigated the most severe vulnerabilities in the custom HMS logic;
- **scope correction**, which ensured that Fortify was no longer evaluating backup and demo material that is not part of the deployed application.

Therefore, the final 0 Critical result is meaningful because it does not come from blind suppression of findings, but from:

- actual secure-coding improvements in authentication, search, state-changing workflows, session handling, and output encoding;
- explicit separation between the runtime application and non-runtime third-party artifacts.

2.1.4 Scope Distinction: Custom Code vs Vendor Code

The Fortify export also reports a significant number of findings in bundled third-party components, especially:

- TCPDF/
- vendor/ckeditor/
- vendor/jquery-file-upload/
- demo/sample assets included in the repository

These findings should be documented as part of the broader security posture of the project, but they should be classified separately from the custom codebase because:

- they do not necessarily reflect design or implementation decisions made by the project team;
- many can be addressed through version upgrades, dependency reduction, or removal of sample/demo files;
- they are less useful than custom findings for demonstrating secure coding remediation on the core HMS logic.

2.1.5 Consolidated SAST Findings for the Case Study

The following table summarizes the most representative and report-worthy findings extracted from Fortify after preliminary triage.

Vulnerability Details and References

Vulnerability	File	CWE	Severity
SQL Injection	func.php	CWE-89	Critical
SQL Injection	func1.php	CWE-89	Critical
SQL Injection	func2.php	CWE-89	Critical
SQL Injection	func3.php	CWE-89	Critical
SQL Injection	contact.php	CWE-89	Critical
SQL Injection	admin-panel.php, admin-panel1.php, prescribe.php	CWE-89	Critical
SQL Injection	patientsearch.php, doctorsearch.php, appsearch.php, messearch.php, search.php	CWE-89	Critical

Vulnerability	File	CWE	Severity
Persistent XSS	doctor-panel.php	CWE-79 / CWE-80	Critical
Persistent XSS	admin-panel.php, admin panel1.php	CWE-79 / CWE-80	Critical
Reflected XSS	prescribe.php	CWE-79 / CWE-80	Critical
Persistent XSS	patientsearch.php, doctorsearch.php, messearch.php, appsearch.php, search.php, newfunc.php, include/header.php	CWE-79 / CWE-80	Critical / High
Cross-Site Request Forgery	multiple form-based workflows	CWE-352	Critical / High
Privacy Violation	patientsearch.php, doctorsearch.php, admin panel1.php	CWE-359	Critical
Privacy Violation: Autocomplete	index.php, index1.php, doctor-panel.php, admin panel1.php	CWE-525	High

Description and Evidence of the Vulnerability Assessment

Vulnerability (Ref)	Description	Evidence
SQL Injection	User-controlled login data reaches SQL query construction in patient authentication logic	Fortify reports SQL Injection at lines such as 8 and 62
SQL Injection	Doctor login logic concatenates untrusted input into SQL statements	Fortify reports SQL Injection in doctor authentication flow
SQL Injection	Registration and update flows use unsanitized input in SQL commands	Fortify reports SQL Injection in registration and state update logic
SQL Injection	Admin authentication and update flows are vulnerable to SQL Injection	Fortify reports SQL Injection in admin login and related operations
SQL Injection	Contact form fields reach SQL insertion without safe parameterization	Fortify reports SQL Injection in contact submission flow
SQL Injection	Sensitive operational actions and data insertion logic are affected by unsafe query construction	Fortify flags multiple SQL Injection instances in core operational panels

Vulnerability (Ref)	Description	Evidence
SQL Injection	Search endpoints interpolate request parameters directly into SELECT queries	Fortify reports SQL Injection across search handlers
Persistent XSS	Application data is rendered in HTML output without safe encoding	Fortify reports multiple Persistent XSS instances in doctor-facing views
Persistent XSS	Administrative data rendering is vulnerable to stored script injection	Fortify reports repeated Persistent XSS findings in admin interfaces
Reflected XSS	Request-driven values are reflected in output contexts without output encoding	Fortify reports multiple Reflected XSS findings in prescription-related flow
Persistent XSS	Search and rendering paths expose untrusted data in browser output	Fortify reports persistent XSS across several rendering endpoints
Cross-Site Request Forgery	Sensitive actions are reachable without anti-CSRF validation	Fortify reports CSRF findings aligned with prior DAST reasoning

Vulnerability (Ref)	Description	Evidence
Privacy Violation	Sensitive user information may be exposed beyond strict need-to-know boundaries	Fortify explicitly reports Privacy Violation findings in user search/admin views
Privacy Violation: Autocomplete	Sensitive form fields may allow insecure browser-side retention through autocomplete	Fortify reports privacy-related autocomplete weaknesses in sensitive forms

2.1.6 Resolution Status of Representative SAST Findings

Classification and Status of Mitigations

Vulnerability Cluster	Representative Files	Current Status
SQL Injection in patient/doctor/admin login	func.php, func1.php, func3.php	Resolved
SQL Injection in registration and contact insertion	func2.php, contact.php	Resolved
SQL Injection in search handlers	search.php, patientsearch.php, doctorsearch.php, appsearch.php, messearch.php	Resolved

Vulnerability Cluster	Representative Files	Current Status
SQL Injection in prescription creation	prescribe.php	Resolved
SQL Injection in admin/patient/doctor operational panels	admin-panel.php, admin-panel1.php, doctor-panel.php, newfunc.php	Further Mitigated
Reflected XSS in prescription flow	prescribe.php	Further Mitigated
Persistent XSS in search/rendering pages	patientsearch.php, doctorsearch.php, appsearch.php, messearch.php, search.php	Further Mitigated
Persistent XSS in main panels	doctor-panel.php, admin panel.php, admin-panel1.php	Partially Mitigated
Cross-Site Request Forgery in main workflows	index.php, index1.php, admin-panel.php, admin-panel1.php, doctor-panel.php, prescribe.php	Further Mitigated
Privacy exposure of password data	patientsearch.php, doctorsearch.php, admin panel1.php	Resolved / Further Mitigated
Privacy Violation / Autocomplete	index.php, index1.php, doctor-panel.php, admin-panel1.php	Partially Mitigated

Vulnerability Cluster	Representative Files	Current Status
Session hardening posture	include/session_bootstrap.php, authentication entry points	Further Mitigated

Technical Details and Remediation Notes

Vulnerability Cluster (Ref)	Notes
SQL Injection (Login)	Replaced vulnerable login queries with prepared statements and password compatibility verification
SQL Injection (Registration/Contact)	Insert logic moved to prepared statements; registration now uses password hashing
SQL Injection (Search)	Search queries parameterized and critical output points encoded
SQL Injection (Prescription)	Prescription insert path rewritten with prepared statements
SQL Injection (Operational Panels)	Core booking, cancellation, billing, doctor-management, and several update flows have been hardened through prepared statements and safer request handling
Reflected XSS (Prescription)	High-risk reflected values are sanitized/encoded; the remaining risk is lower and mostly tied to broader UI review

Vulnerability Cluster (Ref)	Notes
Persistent XSS (Search/Rendering)	Core rendered values now pass through output encoding; residual review remains advisable for secondary rendering paths
Persistent XSS (Main Panels)	High-risk table outputs, reflected values, and several rendered session/user fields have been encoded, but the panel surface remains broad
CSRF (Main Workflows)	CSRF tokens have been introduced in the main sensitive forms and multiple state-changing workflows have been migrated from GET to POST
Privacy Exposure (Passwords)	Passwords are no longer displayed in search results and have been removed or redacted from the most exposed admin-facing views
Privacy Violation / Autocomplete	Login/registration and some admin-facing forms now use stricter autocomplete settings, but a full UI pass is still recommended
Session Hardening	Cookie flags, inactivity timeout, lightweight fingerprinting, and centralized session bootstrap now reduce fixation and hijacking risk

2.1.7 Final Fortify-to-Remediation Mapping

The following table provides a concise final mapping between the most representative Fortify clusters, the remediation work applied in code, and the residual status that should be discussed during the presentation.

Scope and Status of Vulnerabilities

Fortify Finding Cluster	Files	Current Status
SQL Injection in login flows	func.php, func1.php, func3.php	Closed
SQL Injection in registration/contact flows	func2.php, contact.php	Closed
SQL Injection in search endpoints	search.php, patientsearch.php, doctorsearch.php, appsearch.php, messearch.php	Closed
SQL Injection in operational panels	admin-panel.php, admin-panel1.php, doctor-panel.php, newfunc.php, prescribe.php	Mostly Mitigated
Persistent / Reflected XSS in major views	doctor-panel.php, admin-panel.php, admin-panel1.php, prescribe.php	Partially Mitigated
Persistent XSS in search/rendering pages	patientsearch.php, doctorsearch.php, search.php, appsearch.php, messearch.php, newfunc.php	Mostly Mitigated
Cross-Site Request Forgery	index.php, index1.php, admin-panel.php, admin-panel1.php, doctor-panel.php, prescribe.php	Mostly Mitigated

Fortify Finding Cluster	Files	Current Status
Privacy Violation	patientsearch.php, doctorsearch.php, admin panel1.php	Mostly Mitigated
Privacy Violation: Autocomplete	index.php, index1.php, doctor-panel.php, admin panel1.php	Partially Mitigated
Session/Cookie hardening related findings	session entry points and panel bootstraps	Mostly Mitigated

Fix and Residual Risk Interventions

Fortify Finding Cluster (Ref)	Fix Applied	Residual Risk
SQL Injection in login flows	Prepared statements, safer input handling, compatibility-aware password verification, session regeneration	Low residual risk, mainly dependent on future maintenance consistency
SQL Injection in registration/contact flows	Prepared statements, password hashing, generic error handling improvements	Low

Fortify Finding Cluster (Ref)	Fix Applied	Residual Risk
SQL Injection in search endpoints	Parameterized queries and output encoding	Low
SQL Injection in operational panels	Prepared statements in core workflows, safer state updates, reduced reliance on direct request-driven queries	Medium-low, final review still advisable on less central flows
Persistent / Reflected XSS in major views	sse_e(...) output encoding, safer reflected fields, reduced exposure in tables	Medium, due to broad UI surface and the possibility of missed rendering points
Persistent XSS in search/rendering pages	Output encoding on main rendered values and safer query/result handling	Medium-low, with residual review recommended for secondary rendering paths
Cross-Site Request Forgery	CSRF tokens, server-side token validation, conversion of sensitive GET actions to POST	Medium-low, as future new forms must follow the same pattern
Privacy Violation	Password removal/redaction, minimized table output, reduced	Medium-low

Fortify Finding Cluster (Ref)	Fix Applied	Residual Risk
	unnecessary field exposure	
Privacy Violation: Autocomplete	autocomplete="off", new-password, current-password, stricter sensitive-field settings	Medium
Session/Cookie hardening	Centralized session bootstrap with HttpOnly, SameSite, inactivity timeout,fingerprinting, strict cookie settings	Medium-low; stronger HTTPS/header deployment policies would improve this further

2.1.8 Interpretation of SAST Evidence

The Fortify findings reinforce the conclusions already derived from manual inspection and DAST-oriented reasoning. In particular, SAST confirms four structural weakness clusters:

1. **Unsafe SQL handling is systemic**, not isolated to a single endpoint.
2. **Output encoding is insufficient**, leading to both persistent and reflected XSS exposure.
3. **Sensitive information is overexposed**, especially in search and administrative views.
4. **Request integrity protections are weak**, as reflected by CSRF-related findings.

This convergence between SAST and DAST is especially important for the case study, because it shows that the application is vulnerable both in its observable behavior and in its internal implementation structure.

2.1.9 Findings to Treat Separately from the Core Case Study

The Fortify export also includes findings in third-party libraries and sample/demo files, such as:

- TCPDF examples and crypto-related utilities;
- ckeditor sample pages and embedded scripts;
- jquery-file-upload server-side components;
- auxiliary vendor assets not central to the hospital workflows.

These findings should be classified as **dependency-related** or **out-of-scope for direct secure coding remediation in the HMS custom logic**. They are still useful to mention in the report as supply-chain or maintenance risks, but they should not be mixed with the primary evidence used to discuss the secure re-engineering of the core application.

2.1.10 False Positives and Duplication Handling

The Fortify export contains multiple repeated findings on similar sinks and neighboring output lines. This is normal in data-flow-based static analysis and does not imply that each row corresponds to a distinct business vulnerability. Therefore, the analysis should:

- merge duplicated issues by vulnerability type and logical code region;
- treat clusters of repeated XSS warnings in the same view as a single structural output-encoding problem;
- treat repeated SQL Injection findings in the same handler as a recurring unsafe query-construction pattern;
- keep vendor findings separated from core application findings.

2.1.11 Role of Fortify in the completed case study

Within the final version of this case study, Fortify serves as:

- formal evidence for the **SAST subsection** of the report;

- confirmation of vulnerabilities already suspected through manual review and DAST;
- a measurable indicator of the reduction in the attack surface after remediation;
- support for the privacy analysis, especially where Fortify explicitly identifies privacy-relevant exposure patterns.

2.2 Dynamic Code Analysis

The **Dynamic Code Analysis (DAST)** phase constitutes the core of the process of formalizing and classifying application weaknesses. While active reconnaissance focused on detecting and enumerating vulnerable endpoints at runtime, vulnerability analysis aims to examine the mathematical and structural logic of these flaws, formalizing their riskiness in relation to international standards (**OWASP Top 10** and **Common Weakness Enumeration – CWE**) and quantifying their potential impact on the corporate **Risk Management** framework.

From an architectural perspective, analyzing the responses provided by the server allows us to validate the actual exploitability of the flaws within the three-tier infrastructure stack, taking into account the real-world interactions between the **Apache web server**, the **PHP interpreter**, and the **MySQL database engine**.

This phase is consistent with the course notion that a **vulnerability** is an exploitable weakness, while an **exploit** is the concrete mechanism used to abuse it. DAST does not merely catalogue coding mistakes: it helps determine whether a given weakness can be transformed into an attack path against a real asset exposed by the running system.

2.2.1 Detailed Analysis and Classification of Detected Vulnerabilities

We proceed with a taxonomic analysis of the main classes of weaknesses that threaten the Hospital Management System, aggregating the detected alerts into structural macro-categories.

2.2.2 SQL Injection (In-Band & Blind Time-Based)

Standard Identifiers: CWE-89 / OWASP A03:2021 – Injection

Vulnerability Mechanism:

As evidenced by combined alerts on func.php, func1.php, func2.php, and func3.php, the application suffers from a lack of proper separation between control statements and dynamic data entered into form fields. The weakness lies in the use of destructive dynamic concatenation (*string concatenation*), in which user input is directly merged into SQL queries without prior validation or the use of parameterized prepared statements.

CIA Triad Impact Analysis:

- **Confidentiality (Compromised):** allows authentication bypass and exfiltration of patient data.
- **Integrity (Compromised):** allows destructive queries such as UPDATE, DELETE, or DROP TABLE.
- **Availability (Compromised):** allows saturation of DB resources via conditional delays such as SLEEP().

From the course theory, SQL Injection is one of the most impactful examples of **OWASP A03:2021 – Injection** and of **CWE-89**, because it breaks the intended separation between **code** and **data**. Prepared statements are therefore not just an implementation detail, but the standard architectural countermeasure that restores this separation by forcing user input to be treated as data rather than executable SQL syntax.

2.2.3 Lack of Anti-CSRF Tokens

Standard Identifiers: CWE-352 / OWASP A05:2021 – Broken Access Control

Structural Vulnerability:

The lack of validation mechanisms independent of session cookies allows a malicious third-party site to exploit browser automation in attaching active authentication tokens to the originating domain. The server processes the request as legitimate.

Impact on Access Control:

This constitutes a critical vector for account takeover or unauthorized state-changing actions.

The theoretical logic behind CSRF, as discussed in the course, is that the attack exploits the **trust that the application places in the user's browser**. If the application authorizes state-changing requests only on the basis of an authenticated cookie, an attacker can induce the browser to replay those

requests without the user's intention. CSRF tokens, SameSite cookies, and the elimination of state-changing GET operations are therefore all mechanisms that restore explicit request integrity.

2.2.4 Application Error Disclosure

Standard Identifiers: CWE-209 / OWASP A05:2021 – Security Misconfiguration

Vulnerability Mechanism:

Caused by the failure to suppress runtime debug messages. When the application encounters an exception in database-related functions, the PHP engine may generate a fatal error and expose it directly in the HTML response.

Operational Relevance:

This weakness accelerates target mapping by revealing internal paths, stack information, configuration clues, or database account details.

2.2.5 Lack of Security Flags on Session Cookies

Standard Identifiers: CWE-1004 / OWASP A02:2021 – Cryptographic Failures

Combined Risk Analysis:

The lack of the HttpOnly flag allows programmatic access to cookies via browser scripts. In combination with a latent **XSS** flaw, the weakness may enable session exfiltration and hijacking.

This also links directly to the theory of **access control**: authentication is only one phase in the control process, and weak session management undermines the entire chain of identification, authentication, authorization, and accounting. In other words, even correct login credentials lose their value if the resulting session can be stolen, replayed, or fixed by an attacker.

2.2.6 Risk Assessment Summary

In line with **ISO/IEC 27005**, application risk is calculated by cross-referencing the probability of exploiting a flaw and its impact on protected assets.

The DAST analysis indicates a very high likelihood of compromise due to the apparent absence of proactive controls such as strong validation, robust session hardening, or security headers.

Since the combined impact of **SQL Injection** and **CSRF** undermines confidentiality, integrity, and accountability, the overall risk level of the application can be classified as **Critical**, requiring immediate architectural review.

2.2.7 Consolidated Vulnerability Matrix

The following table consolidates the main weaknesses currently supported by repository inspection and by the DAST-oriented reasoning developed in the previous sections. The matrix is designed to serve as the operational bridge between reconnaissance, vulnerability classification, and later remediation.

Classification and Impact of Vulnerabilities

Vulnerability	Endpoint / File	CWE	OWASP Top 10	CIA Impact
SQL Injection in patient login	func.php	CWE-89	A03:2021 – Injection	C / I / A
SQL Injection in doctor login	func1.php	CWE-89	A03:2021 – Injection	C / I / A
SQL Injection in admin login	func3.php	CWE-89	A03:2021 – Injection	C / I / A
SQL Injection in patient registration...	func2.php, contact.php, admin-panel.php	CWE-89	A03:2021 – Injection	C / I / A
SQL Injection in search endpoints	patientsearch.php , doctorsearch.php,	CWE-89	A03:2021 – Injection	C / I

Vulnerability	Endpoint / File	CWE	OWASP Top 10	CIA Impact
	appsearch.php, messearch.php, search.php			
Missing Anti-CSRF protection	index.php flows, admin-panel.php, doctor-panel.php, operational forms	CWE-352	A05:2021 – Broken Access Control	I
Forced state change via GET parameters	admin-panel.php, doctor-panel.php	CWE-352 / CWE-639	A01:2021 – Broken Access Control / A05:2021	I
Potential IDOR / authorization weakness	admin-panel.php, doctor-panel.php, prescribe.php	CWE-639	A01:2021 – Broken Access Control	C / I
Verbose error disclosure	include/config.php, admin-panel.php, doctor-panel.php	CWE-209	A05:2021 – Security Misconfiguration	C
Plaintext password storage and comparison	func.php, func1.php, func2.php, func3.php, admin-facing views	CWE-256 / CWE-312	A02:2021 – Cryptographic Failures	C

Vulnerability	Endpoint / File	CWE	OWASP Top 10	CIA Impact
Insecure session management posture	multiple files using session_start()	CWE-1004	A02:2021 – Cryptographic Failures	C / I
Security misconfiguration / weak deployment...	include/config.php, static resource structure	CWE-16 / CWE-209	A05:2021 – Security Misconfiguration	C / I

Technical Evidence and Risk Assessment

Vulnerability (Ref)	Technical Evidence	Risk	False Positive
SQL Injection (Patient Login)	Query built as select * from patreg where email='$email' and password='$password'; using unsanitized $_POST input	Critical	No
SQL Injection (Doctor Login)	Query built as select * from doctb where username='$dname' and password='$dpass';	Critical	No

Vulnerability (Ref)	Technical Evidence	Risk	False Positive
SQL Injection (Admin Login)	Query built as select * from admintb where username='$username' and password='$password';	Critical	No
SQL Injection (Registration/Insertion)	Multiple INSERT statements concatenate raw $_POST values directly into SQL commands	Critical	No
SQL Injection (Search Endpoints)	Request-derived search parameters are interpolated directly into SELECT queries	High	No
Missing Anti-CSRF protection	State-changing requests are processed without visible CSRF tokens or synchronizer mechanisms	High	Likely True Positive
Forced state change via GET	Appointment cancellation is triggered through ?ID=...&cancel=... and handled server-side without visible anti-forgery protection	High	No

Vulnerability (Ref)	Technical Evidence	Risk	False Positive
Potential IDOR / Auth Weakness	Direct use of request identifiers such as $_GET['ID'] and patient-related query-string values without clear object-level authorization validation	High	Likely True Positive
Verbose error disclosure	Direct output of mysqli_connect_error() and mysqli_error($con) may reveal internal backend information	Medium / High	No
Plaintext password storage	Credentials are compared directly against database values and patient registration stores raw passwords without hashing	Critical	No
Insecure session management	Session handling is widespread and current evidence suggests missing hardening controls such as	High	Likely True Positive

Vulnerability (Ref)	Technical Evidence	Risk	False Positive
	HttpOnly/Secure cookie protection		
Security misconfiguration	Development-style DB configuration (root, empty password, direct error reporting) and publicly browsable static directories were already flagged in prior analysis	High	Likely True Positive

2.2.8 Interpretation of the Matrix

The matrix shows that the application is not affected by a single isolated flaw, but by a cluster of interacting weaknesses concentrated around four structural themes:

1. **Injection vulnerabilities**, especially SQL Injection, across authentication, registration, search, and data persistence flows;
2. **Broken access control patterns**, including missing CSRF protection and likely unsafe object reference handling;
3. **Cryptographic and credential management failures**, due to plaintext password handling and weak session protection;
4. **Security misconfiguration and information disclosure**, which reduce attacker uncertainty and facilitate exploitation.

2.2.9 False Positive Handling

The findings related to raw SQL concatenation, plaintext password handling, verbose error disclosure, and GET-based state changes should be considered **high-confidence true positives**, because they are directly observable in the source code. By contrast, a subset of the broader DAST findings—especially

those related to session hardening and full authorization bypass—should be interpreted more cautiously unless corroborated by explicit runtime evidence or scanner output.

This distinction is important for the case study because it demonstrates methodological rigor: findings directly provable from code inspection can already be classified confidently, while runtime-dependent findings should be confirmed through ZAP sessions, manual browser testing, or Fortify correlation.

2.3 Security Fix

2.3.1 Objectives

- Identify vulnerabilities according to the **OWASP Top 10 Web** and privacy vulnerabilities
- Identify **false positives**
- Provide a possible solution to fix the identified issues (**code and/or description**)

2.3.2 Remediation Strategy

The security remediation phase should follow a **risk-based prioritization approach**, addressing first the vulnerabilities that most severely affect the confidentiality, integrity, and availability of patient-related data and hospital workflows. Based on the findings documented in the DAST-oriented analysis, remediation should be structured around four priorities:

1. **Eliminate injection vectors** in authentication, registration, search, and data persistence logic;
2. **Protect authentication and session management**, especially with respect to password storage and session cookie hardening;
3. **Enforce request integrity and access control**, particularly for CSRF-sensitive operations and object-level authorization checks;
4. **Reduce information leakage and improve secure defaults**, including error handling, headers, and exposure minimization.

This remediation order reflects the security engineering principle that defensive measures should be layered according to **risk reduction value**. In other words, the case study first targets flaws that enable direct compromise

of assets (such as SQL Injection and plaintext credential handling), then addresses controls that improve trust boundaries, request integrity, privacy minimization, and long-term defensive resilience.

2.3.3 Consolidated Remediation Matrix

Issue	Root Cause	Proposed Fix	Expected Benefit
SQL Injection *(login, search, registration, contact, booking)*	Dynamic query concatenation with raw $_POST / $_GET input.	Replace vulnerable mysqli_query() with **prepared statements** (mysqli_prepare() / bind_param()); validate and normalize input before execution.	Prevent authentication bypass, unauthorized data extraction, data tampering, and time based inference.
Plaintext password storage *and direct comparison*	Passwords are stored and compared in raw form.	Store credentials using **password_hash()** and verify them with **password_verify()**; remove plaintext password exposure from admin views.	Reduce impact of credential leaks and align authentication handling with modern secure practices.
Missing anti CSRF protections	State-changing	Introduce **CSRF tokens** for forms and sensitive	Prevent forged actions by

Issue	Root Cause	Proposed Fix	Expected Benefit
	requests rely only on session cookies.	operations; validate tokens server-side before processing requests.	authenticated victims and strengthen workflow integrity.
Forced state changes *via GET parameters*	Sensitive actions (e.g., cancellation) are triggered by URL parameters.	Convert destructive or state-changing actions from **GET to POST**; require CSRF validation and ownership/role checks before execution.	Reduce abuse of links, request forgery, and unauthorized record manipulation.
Potential IDOR / *weak authorization checks*	Direct use of request identifiers without strict object-level authorization .	Verify that each requested resource belongs to the authenticated user; implement **server-side access control checks** per object.	Prevent cross-user access, unauthorized cancellation, and exposure of other patients' or doctors' records.
Verbose error disclosure	Runtime database errors are	Disable detailed error output in production; use **centralized**	Reduce reconnaissance support for attackers and

Issue	Root Cause	Proposed Fix	Expected Benefit
	echoed directly to the client.	**exception handling** and server-side logging instead of direct browser disclosure.	limit backend information leakage.
Weak session hardening	Widespread session usage without visible secure cookie controls.	Configure session cookies with **HttpOnly, Secure, and SameSite**; regenerate session IDs on login; invalidate sessions properly on logout.	Reduce risk of session hijacking, fixation, and replay.
Security misconfiguration	Development-style database config, weak defaults, exposed static structures.	Replace development credentials, enforce **least-privilege DB accounts**, disable directory listing, and remove unnecessary public exposure.	Improve baseline hardening and reduce the attack surface.

Issue	Root Cause	Proposed Fix	Expected Benefit
Sensitive data overexposure *in views*	Admin and operational panels expose excessive credential or user information.	Remove password fields from UI, limit displayed attributes to operationally necessary ones, and **minimize sensitive data** in logs/reports.	Improve confidentiality and support privacy-by-design principles.
Missing browser-side *hardening headers*	No visible CSP, clickjacking protection, or MIME sniffing defenses.	Add security headers such as **Content-Security-Policy**, X-Frame-Options, X Content-Type-Options, and Referrer-Policy.	Strengthen client-side protection against browser-based attack vectors.

2.3.4 Priority-Based Remediation Plan

To maximize the value of both the case study and the practical code-fixing effort, remediation should be performed in the following order:

2.3.4.1 Priority 1 — Critical Fixes

- SQL Injection elimination;
- password hashing and secure authentication flow redesign;
- removal of plaintext password exposure;

- suppression of verbose database errors.

These fixes have the highest impact because they directly reduce the risk of full compromise, credential theft, and sensitive data exposure.

2.3.4.2 Priority 2 — High-Value Integrity Protections

- CSRF token introduction;
- conversion of state-changing GET actions into POST-based protected workflows;
- object-level authorization checks for appointment, prescription, and user-related records.

These changes primarily protect **integrity** and help prevent unauthorized actions in authenticated contexts.

2.3.4.3 Priority 3 — Hardening and Resilience Improvements

- session cookie hardening;
- session regeneration after login;
- security headers;
- removal of weak deployment defaults.

These measures improve the overall defensive posture and reduce exploit chaining opportunities.

2.3.5 Example Remediation Directions for the Codebase

The codebase can be improved with concrete implementation patterns such as the following:

2.3.5.1 Replace Vulnerable Login Queries with Prepared Statements

Instead of code patterns such as:

```
$query="select * from patreg where email='$email' and
password='$password';";
$result=mysqli_query($con,$query);
```

the application should move toward logic such as:

```php
$stmt = mysqli_prepare($con, "SELECT pid, fname, lname, gender, contact,
email, password FROM patreg WHERE email = ?");
mysqli_stmt_bind_param($stmt, "s", $email);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
```

combined with password_verify() for password comparison.

2.3.5.2 Protect Password Storage

During registration, instead of inserting raw passwords:

```php
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
```

and only the hash should be stored in the database.

2.3.5.3 Protect State-Changing Forms with CSRF Tokens

Each sensitive form should:

- generate a per-session token;
- include it as a hidden field;
- validate it server-side before processing any update, insert, delete, or cancel operation.

2.3.5.4 Remove Verbose Error Messages from Responses

Instead of exposing database errors directly to end users, the application should:

- log errors server-side;
- return a generic message to the client;
- keep detailed debugging information out of production responses.

2.3.6 Example Bug-Fix Snippets by Vulnerability Cluster

Below are representative examples of the kinds of source-code fixes applied or recommended for the main vulnerability clusters.

2.3.6.1 Cluster A — SQL Injection

Example A1 — vulnerable login query before remediation

```
$query="select * from patreg where email='$email' and
password='$password';";
$result=mysqli_query($con,$query);
```

Example A1 — safer version after remediation

```
$stmt = mysqli_prepare($con, "SELECT pid, fname, lname, gender, contact,
email, password FROM patreg WHERE email = ? LIMIT 1");
mysqli_stmt_bind_param($stmt, "s", $email);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
```

Example A2 — vulnerable insert before remediation

```
$query="insert into contact(name,email,contact,message)
values('$name','$email','$contact','$message');";
$result = mysqli_query($con,$query);
```

Example A2 — safer version after remediation

```
$stmt = mysqli_prepare($con, "INSERT INTO
contact(name,email,contact,message) VALUES(?,?,?,?)");
mysqli_stmt_bind_param($stmt, "ssss", $name, $email, $contact,
$message);
$result = mysqli_stmt_execute($stmt);
```

2.3.6.2 Cluster B — Password Management / Cryptographic Failures

Example B1 — insecure password storage before remediation

```
$query="insert into
patreg(fname,lname,gender,email,contact,password,cpassword) values
('$fname','$lname','$gender','$email','$contact','$password','$cpassword');
```

Example B1 — safer version after remediation

```php
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
$stmt = mysqli_prepare($con, "INSERT INTO
patreg(fname,lname,gender,email,contact,password,cpassword) VALUES
(?,?,?,?,?,?,?)");
mysqli_stmt_bind_param($stmt, "sssssss", $fname, $lname, $gender,
$email, $contact, $hashedPassword, $hashedPassword);
```

Example B2 — plaintext comparison before remediation

```php
if($row['password'] == $password) {
    // authenticated
}
```

Example B2 — compatibility-aware verification after remediation

```php
if (sse_verify_password_compat($password, $row['password'])) {
    sse_regenerate_session();
    // authenticated
}
```

2.3.6.3 Cluster C — XSS / Unsafe Output Rendering

Example C1 — unsafe output before remediation

```php
echo "<td>$email</td>";
```

Example C1 — safer output after remediation

```php
echo "<td>".sse_e($email)."</td>";
```

Example C2 — unsafe reflected hidden field before remediation

```php
<input type="hidden" name="fname" value="<?php echo $fname ?>" />
```

Example C2 — safer reflected hidden field after remediation

```php
<input type="hidden" name="fname" value="<?php echo sse_e($fname) ?>" />
```

2.3.6.4 Cluster D — Privacy Overexposure

Example D1 — overexposed patient search result before remediation

```
<th scope='col'>Password</th>
...
<td>$password</td>
```

Example D1 — minimized result after remediation

```
<th scope='col'>First Name</th>
<th scope='col'>Last Name</th>
<th scope='col'>Email</th>
<th scope='col'>Contact</th>
```

Example D2 — over-broad doctor search before remediation

```
$query = "select * from doctb where email= '$contact'";
```

Example D2 — minimized query after remediation

```
$stmt = mysqli_prepare($con, "SELECT username, email, docFees FROM doctb
WHERE email = ? LIMIT 1");
```

2.3.7 Relationship with False Positives

Not all scanner alerts should automatically lead to code modifications. The remediation phase must distinguish between:

- **confirmed flaws**, directly observable in the source code and requiring real fixes;
- **runtime-dependent findings**, which should be validated with ZAP or Fortify before applying invasive changes;
- **contextual false positives**, where the reported weakness may be partially mitigated by logic not immediately visible in a single endpoint.

In this case study, the SQL Injection patterns, plaintext password handling, verbose error disclosure, and unsafe state-changing request handling are treated as **true positives requiring remediation**.

2.3.8 Expected Outcome of the Remediation Phase

If the proposed fixes are applied consistently, the Hospital Management System would benefit from:

- stronger resistance to OWASP Top 10 injection attacks;
- improved protection of credentials and sessions;
- better integrity controls over appointments and administrative operations;
- reduced information leakage and improved defensive hardening;
- better alignment with privacy-by-design and secure software engineering principles.

This remediation layer therefore represents the bridge between vulnerability discovery and the architectural re-engineering of the system into a more secure and privacy-aware platform.

2.3.9 Security Fix Applied / Residual Risk

The remediation work performed on the codebase was implemented through four incremental hardening blocks, each designed to reduce the highest-value risks first and then progressively improve the exposed operational panels.

Block 1 — Critical injection and authentication remediation

- prepared statements introduced in the main login flows (func.php, func1.php, func3.php);
- safer handling of user-controlled authentication inputs;
- initial session regeneration on successful authentication;
- creation of a shared helper layer for normalization, output encoding, and password verification.

Block 2 — Registration, search, contact, and prescription remediation

- prepared statements introduced in registration, search, contact, and prescription-related endpoints;

- password hashing introduced for new credentials;
- output encoding added to key rendering points;
- privacy overexposure reduced by removing password display from sensitive result pages.

Block 3 — Session hardening, CSRF, and panel-level XSS reduction

- centralized session bootstrap introduced with cookie hardening, timeout handling, and a lightweight session fingerprint;
- CSRF tokens added to the main sensitive forms;
- doctor- and admin-facing panels hardened through output encoding and safer state-changing operations.

Block 4 — Final hardening of residual operational workflows

- patient- and doctor-side appointment cancellations migrated from GET to POST with CSRF validation;
- bill-generation workflow hardened with POST and CSRF validation;
- additional privacy and rendering cleanup applied to admin-panel.php and admin-panel1.php;
- final review of newfunc.php confirmed that the most exposed residual flows were already aligned with the remediation strategy.

Residual risk after code remediation

Even after the four remediation blocks, some residual risk remains, which should be explicitly acknowledged in the case study:

- **large rendering surfaces** in major panels may still hide isolated XSS-prone output points not yet reviewed exhaustively;
- **object-level authorization / IDOR-like scenarios** require deeper functional validation beyond input sanitization and CSRF hardening;
- **browser and deployment hardening** (for example CSP, X-Frame-Options, X-Content-Type-Options, HTTPS-only production deployment, and least-privilege database configuration) are still recommended;
- the latest Fortify rerun already confirms the elimination of residual critical findings after the final scope correction, while additional future scans may still be useful for incremental hardening.

Overall, however, the application has moved from a state dominated by critical injection and credential-handling flaws to a more defensible baseline in which the remaining issues are narrower, more localized, and more suitable for a final hardening pass rather than broad structural emergency remediation.

Manual security regression tests on the remediated localhost deployment

To complement static analysis and source-code review, a small set of **manual regression tests** was executed on the localhost deployment of the application, comparing the original vulnerable behavior with the behavior observed after remediation. These tests are especially useful because they provide runtime evidence that the login flows previously affected by SQL Injection no longer accept attacker-controlled SQL payloads as authentication bypass vectors.

Test case 1 — Administrative login SQL Injection bypass

Objective: verify whether the administrative authentication flow is vulnerable to SQL Injection-based login bypass.

Payload used:

- username: ' OR '1'='1
- password: arbitrary value

Observed result before remediation: the original version of the application allowed successful access to the administrative area even when the username field contained the SQL Injection payload.

Observed result after remediation: in the remediated version, authentication no longer succeeds, the administrative panel is not reached, and no SQL error is displayed to the user.

Interpretation: this behavior is consistent with the replacement of vulnerable dynamic SQL concatenation with parameterized query handling in the administrative login flow.

Test case 2 — Doctor login with comment-based SQL payload

Objective: verify whether the doctor authentication flow can be bypassed using a comment-based SQL Injection payload.

Payload used:

- Username: ashok' -- -
- Password: test

Observed result before remediation: in the original version of the application, the payload allowed successful authentication and access to the doctor-facing area.

Observed result after remediation: in the remediated version, login no longer succeeds and access to the doctor area is denied.

Interpretation: this indicates that the login flow is no longer interpreting attacker-supplied input as executable SQL syntax, which is coherent with the adoption of prepared statements in the doctor authentication logic.

Test case 3 — Session fixation validation

Objective:verify whether the application reuses the same session identifier before and after authentication, which would expose the system to session fixation.

Procedure used:

- the browser session cookie (PHPSESSID) was observed before login using browser developer tools;
- authentication was then performed using a valid account;
- the session identifier was checked again immediately after successful login.

Observed result before remediation:the original application did not guarantee robust session lifecycle management and was therefore exposed to weaker session handling patterns.

Observed result after remediation:the remediated application regenerates the session after successful authentication through centralized secure session handling.

Interpretation:this is consistent with the use ofsse_regenerate_session()and with the strengthened session bootstrap logic introduced during remediation.

Test case 4 — Post-logout direct access to protected pages

Objective: verify whether a user can still access a protected page directly via URL after performing logout.

Procedure used:

- login was performed into a protected area;
- the URL of the protected page was copied;
- logout was executed;
- the protected URL was pasted again directly into the browser address bar.

Observed result before remediation: protected pages could still be reached or partially displayed after logout, indicating incomplete session invalidation and/or insufficient access checks at page entry.

Observed result after remediation: direct access to protected pages after logout is blocked and the user is redirected to the appropriate public login/home page.

Interpretation: the fix is consistent with the introduction of stronger logout logic, explicit destruction of session data and cookies, centralized authentication guards, and anti-cache protections.

Test case 5 — Reflected XSS probe on authentication/input flows

Objective: verify whether attacker-controlled HTML/JavaScript input is reflected back into the response without output encoding.

Payload used:

<script>alert('XSS')</script>

Observed result before remediation: the original application exposed multiple output-handling weaknesses and was structurally compatible with reflected XSS risk in input-driven views.

Observed result after remediation: the payload is not executed as browser script in the tested remediated flows.

Interpretation: this is coherent with the introduction of output encoding and safer rendering practices in the remediated application.

Test case 6 — IDOR / object-level authorization probe

Objective:verify whether changing record identifiers in request parameters allows access to resources belonging to other users.

Procedure used:

- request parameters such as appointment or patient-related identifiers were manually altered in accessible workflows;
- the goal was to determine whether the system would expose or act on resources outside the legitimate authorization scope.

Observed result before remediation:the original design exposed GET-driven identifiers and therefore presented an IDOR-like risk surface.

Observed result after remediation:in the tested scenarios, the remediated application did not allow successful exploitation through simple identifier manipulation.

Interpretation:the result is consistent with the reduction of unsafe GET workflows and with the improved handling of sensitive actions, even though deeper object-level authorization review remains a residual-risk area in the report.

Consolidated summary of the executed manual SQL Injection regression tests

Test Case	Target	Payload	Before Remediation	After Remediation	Outcome
SQL Injection login bypass	Admin login	' OR '1'='1	Login succeeded	Login denied	**Passed**
Comment-based SQL Injection payload	Doctor login	ashok' -- -- / test	Login succeeded	Login denied	**Passed**

These regression tests should not be interpreted as a mathematical proof that all SQL Injection risk has been eliminated from the entire project. However, they do provide strong practical evidence that two previously exploitable authentication flows were successfully hardened and that the applied remediation has produced observable security improvement at runtime.

- **Traceability of implementation choices and scan-scope reduction**

In addition to the code-level fixes, the remediation activity included a deliberate effort to improve the **traceability of implementation choices**, distinguish **runtime components** from **non-deployed materials**, and reduce scanner noise originating from backup or demo artifacts. This decision is especially relevant from an academic Secure Software Engineering perspective, because the final evaluation should focus on the **actual attack surface of the deployable Hospital Management System**, rather than on legacy copies or sample files that are not used by the application.

The following implementation choices were applied and should be explicitly documented as part of the remediation rationale.

A. Application-level hardening choices applied in code

The remediation process introduced multiple secure-coding and hardening measures directly in the custom HMS codebase:

- **prepared statements** were introduced in the main authentication, registration, search, contact, and prescription-related flows to reduce SQL Injection exposure;
- **password hashing / compatibility-aware verification** was introduced for newly managed credentials, replacing insecure plaintext-oriented handling in the most exposed flows;
- **output encoding** was expanded in the administrative and doctor-facing panels to reduce XSS-prone rendering paths;
- **CSRF protections** were added to sensitive forms and state-changing workflows;
- **state-changing GET operations** were progressively migrated toward safer POST-based handling with CSRF validation;

- **session handling** was centralized through include/session_bootstrap.php, introducing cookie hardening, inactivity timeout controls, and a lightweight session fingerprint;
- **privacy minimization** was improved by removing unnecessary password exposure and reducing overly broad rendering of sensitive values in administrative views;
- **TCPDF integration** used for bill generation was preserved, but the surrounding dependency footprint was reviewed to keep only the functionality effectively used by the HMS runtime.

These choices are important because they show that the remediation activity did not simply suppress findings, but instead modified the application logic to reduce exploitability and align the code with secure development principles.

B. Files and directories removed from the effective scan scope

During the remediation phase, several directories were found to generate severe or misleading Fortify findings even though they were **not part of the runtime path actually exercised by the HMS**. In the local project copy used for remediation, they were moved under _scan_excluded/ in order to keep a traceable copy while excluding them from the active scan perimeter.

The following artifacts were removed from the effective scan scope:

Artifact moved out of active scan scope	Reason for exclusion
TCPDF_backup_preupdate/	Legacy backup of the previous TCPDF version (6.3.5), retained only as rollback material and not referenced by the running HMS
TCPDF/examples/	Vendor demo material, including example PDFs, sample workflows, and example certificates not used by the application runtime

Artifact moved out of active scan scope	Reason for exclusion
TCPDF/test/	Vendor unit/integration test material not executed by the HMS application
TCPDF/scripts/	Vendor utility scripts and smoke examples not part of the deployed web workflow
TCPDF/.git and TCPDF/.github	Dependency source-control metadata and CI configuration, irrelevant to runtime security evaluation

This choice was driven by direct evidence from the codebase: the HMS custom code references only TCPDF/tcpdf.php for PDF generation, while the excluded folders were not used by the deployed business workflow.

C. Why these exclusions are technically justified

The scope reduction is technically justified for three reasons:

1. **Runtime relevance**: the excluded materials are not referenced by the active HMS flows and therefore do not belong to the effective runtime attack surface.
2. **Risk attribution correctness**: critical and high-severity findings generated on backup copies, example certificates, tests, or utility scripts would otherwise be incorrectly attributed to the core HMS implementation.
3. **Supply-chain clarity**: the approach preserves visibility over third-party risks while avoiding confusion between:
    - vulnerabilities in **custom HMS logic**;
    - vulnerabilities in **legacy backup copies**;
    - vulnerabilities in **vendor sample/demo assets** that are not deployed.

D. Academic justification for excluding these files from the scan perimeter

From an academic point of view, excluding these files is appropriate **only because they are not part of the final deployable system**. The goal of the case study is not to artificially reduce the number of findings, but to ensure that the analysis reflects the **real security posture of the application being delivered**.

The academically correct interpretation is therefore:

the scan perimeter was reduced to the code, libraries, and configuration elements effectively used by the Hospital Management System at runtime, while backup copies, vendor demo files, tests, and support scripts were preserved separately for traceability but excluded from the operational security assessment.

This distinction is methodologically important because it avoids two opposite mistakes:

- **over-reporting**, where the system appears more vulnerable than it actually is due to non-runtime artifacts;
- **under-reporting**, which would happen if actively used components were excluded without justification.

E. Residual limitations of the scan after scope reduction

Even after this cleanup, some findings may still remain for reasons that should be acknowledged transparently:

- Fortify may remain conservative on some **output-encoding / XSS poor validation** paths even after safe rendering changes;
- session-related findings may persist if the scanner cannot fully infer the deployment assumption of HTTPS and hardened cookie delivery;
- third-party runtime code such as the active TCPDF core may still contain low-level or dependency-related findings that are not directly attributable to custom HMS development.

For this reason, the final Fortify rerun should be interpreted as a more accurate picture of the **real residual risk of the deployable system**, rather than as an absolute elimination of all theoretical scanner alerts.

3. PRIVACY ANALYSIS

3.1 Privacy Assessment

The privacy assessment phase evaluates the Hospital Management System not only as a vulnerable web application, but as a system that processes **personal data** and **health-related operational information** whose misuse may produce legal, ethical, and organizational consequences. In this context, privacy cannot be treated as a secondary concern to security: it must be addressed as a design property of the system, in line with the principles of **Privacy by Design**, the **Privacy Knowledge Base (PKB)**, and GDPR-oriented data protection requirements.

From the code inspection, from the DAST/SAST evidence, and from the Fortify findings explicitly tagged as **Privacy Violation** or **Privacy Violation: Autocomplete**, the application shows multiple privacy weaknesses related to excessive data exposure, over-retention of credentials, and insufficient control over who can view sensitive information.

This framing is fully consistent with the theoretical material on **Privacy Oriented Software Development (POSD)**. In the backward/re-engineering version of POSD, privacy analysis starts from the vulnerabilities identified in the existing system and uses them as input for selecting the **Privacy by Design principles**, **privacy design strategies**, and **privacy patterns** required to redesign the target architecture.

3.1.1 Data Categories Processed by the HMS

The application processes several categories of information that are privacy-relevant and, in part, highly sensitive:

1. **Patient identity data**
    - first name and last name;
    - gender;
    - patient identifier (pid).
2. **Contact and account data**
    - email addresses;
    - phone/contact numbers;

- usernames;
- passwords and session-linked identity attributes.

3. **Appointment and operational data**
   - doctor name;
   - appointment identifiers;
   - dates and times;
   - payment-related status.
4. **Medical and quasi-clinical data**
   - disease descriptions;
   - allergy data;
   - prescriptions.
5. **Feedback and communication data**
   - contact form messages;
   - user-submitted support or feedback content.

Taken together, these elements show that the platform processes both **PII** and **PHI-like information**, making privacy assessment essential even when the application is framed primarily as a software security case study.

3.1.2 Privacy Risks Identified from Code and Fortify Evidence

The main privacy risks currently emerging from the system are the following:

1. Excessive data exposure in administrative and search interfaces

Several pages expose broad record sets through search or listing logic, including patient, doctor, appointment, and contact information. In some cases, administrative panels render fields such as passwords or other unnecessary account attributes. Fortify also reports **Privacy Violation** findings in pages such as patientsearch.php, doctorsearch.php, and admin-panel1.php.

This indicates a violation of the **data minimization** and **need-to-know** principles: users or administrators may be shown more data than is operationally necessary for their specific task.

2. Credential overexposure and insecure storage

The application stores and renders passwords in plaintext-related flows, and credentials are compared directly against database values. Beyond being a security flaw, this is also a privacy problem because credentials are personal authentication secrets and should not be stored or displayed in human-readable form.

3. Over-collection and unrestricted propagation of patient-related data

The code propagates patient identity, contact information, appointment details, disease data, allergy data, and prescriptions across multiple panels and query flows. In the absence of strong access control and purpose limitation, this broad circulation increases the risk of improper disclosure.

4. Client-side privacy leakage through autocomplete and browser retention

Fortify explicitly flags **Privacy Violation: Autocomplete** findings in several forms. This suggests that sensitive fields may be cached or suggested by the browser in contexts where local retention should instead be minimized, especially for passwords or identity-related inputs.

5. Insufficient purpose limitation and display minimization

The system appears to use the same broad data structures across listing, search, and management interfaces, without a clear distinction between:

- data strictly needed for scheduling;
- data needed for account administration;
- data needed for medical workflow support;
- data that should remain hidden unless strictly required.

This creates privacy risk through **function creep**, where data collected for one purpose becomes visible in unrelated operational contexts.

3.1.3 Privacy Design Strategies Selected from the Privacy Knowledge Base

Based on the current architecture and observed weaknesses, the most relevant privacy design strategies for re-engineering the HMS are the following.

The choice of these strategies directly follows the structure of the **Privacy Knowledge Base** discussed in the course. In that model, privacy engineering is not reduced to generic legal compliance statements; instead, it is operationalized by connecting:

- observed vulnerabilities,
- violated Privacy by Design principles,
- privacy design strategies,
- and finally concrete privacy patterns.

3.1.3.1 Minimize

The system should reduce the quantity of personal and medical data collected, stored, and displayed to the minimum required for the intended function.

In practice, this means:

- removing password visibility from admin panels;
- limiting search results to strictly relevant fields;
- avoiding unnecessary replication of patient data across multiple views.

3.1.3.2 Hide

Sensitive data should be protected from unnecessary exposure through encryption, hashing, pseudonymization where appropriate, and restrictive rendering policies.

In practice, this means:

- hashing passwords;
- avoiding plaintext display of sensitive fields;
- limiting direct exposure of disease/allergy/prescription data to strictly authorized roles.

3.1.3.3 Separate

Different categories of information should be separated according to purpose, role, and operational necessity.

In practice, this means:

- separating account management data from medical workflow data;
- separating administrative functions from doctor-facing and patient-facing data views;
- applying object-level access boundaries.

3.1.3.4 Inform

Users should be informed about what data is collected, why it is processed, and how it is used.

In the current project this is largely absent. A re-engineered version should provide:

- privacy notices;
- purpose statements near forms;
- retention and usage explanations.

3.1.3.5 Control

Users and authorized operators should only be able to access and manipulate data according to their legitimate role and business need.

This strategy directly relates to:

- limiting admin overexposure;
- restricting patient search visibility;
- preventing cross-user access to appointments and prescriptions.

3.1.3.6 Enforce

Privacy rules should not remain policy-level statements only; they must be implemented through technical controls.

Examples include:

- role-based access control;
- field-level restriction;
- input validation and output encoding;
- secure session handling.

3.1.3.7 Demonstrate

The system should be able to demonstrate compliance and accountability through evidence and traceability.

This implies the need for:

- audit logging for access to sensitive functions;
- logging of privileged actions;

- traceable handling of administrative operations.

3.1.4 Mapping Privacy Strategies to Privacy Patterns

The following table maps the most relevant privacy strategies to candidate privacy patterns applicable to the Hospital Management System.

Privacy Strategy	Possible Pattern	Application to HMS
Minimize	Data Minimization	Limit stored and displayed patient/account data to fields strictly required by each use case.
Hide	Use of Hashes / Confidential Data Storage	Protect passwords and sensitive identifiers through secure storage and non-display.
Hide	Pseudonymous Identifier	Reduce unnecessary exposure of direct identity attributes where record identifiers are sufficient.
Separate	Role-Based Access Control	Separate patient, doctor, and admin visibility over records and functions.
Inform	Privacy Notice	Inform users about what data is collected in registration, booking, and contact flows.
Control	User-Controlled Access / Least Privilege	Restrict who can view prescriptions, appointments, contact records, and user details.

Privacy Strategy	Possible Pattern	Application to HMS
Enforce	Sticky Policy / Policy Enforcement Point	Ensure privacy constraints are enforced in server-side application logic.
Demonstrate	Audit Log	Track administrative searches, access to patient data, and modifications to appointments or prescriptions.

3.1.5 Privacy Assessment Outcome

The assessment shows that the current HMS design does not yet embody privacy-by-design principles in a mature way. The main deficiencies are:

- overexposure of personal and medical data in interfaces;
- insufficient separation of duties and visibility boundaries;
- insecure management of credentials and sensitive fields;
- lack of user information and transparency mechanisms;
- insufficient technical enforcement of privacy constraints.

However, the system is still a good candidate for privacy-aware re-engineering because the weaknesses are structurally identifiable and can be addressed through a combination of:

- secure coding fixes;
- interface/data minimization;
- access-control redesign;
- architectural privacy patterns.

3.1.6 Link with Fortify and the Next Privacy Step

The Fortify findings of type **Privacy Violation** and **Privacy Violation: Autocomplete** strengthen the case for treating privacy as a first-class

architectural concern. They also provide formal support for the transition from privacy assessment to the next phase, namely the definition of a **Privacy Architecture** that operationalizes the selected strategies in a more secure target system.

3.2 Privacy Architecture

The privacy architecture phase defines the **target architectural model** for transforming the current Hospital Management System into a platform that is not only more secure, but also more compliant with privacy-by-design principles. While the privacy assessment identifies the main weaknesses of the current system, the privacy architecture describes how the application should be reorganized so that privacy requirements become embedded into system structure, data flows, and operational controls.

The goal is not merely to add isolated privacy countermeasures, but to re-engineer the application so that:

- only necessary data is processed;
- visibility is constrained by role and purpose;
- sensitive information is protected at rest, in transit, and in presentation layers;
- access and actions are accountable;
- user data handling becomes consistent with GDPR-oriented expectations.

This is coherent with the **POSD backward process** presented in the course: after the security and privacy assessment, the team derives a **Target Architecture** whose purpose is to integrate privacy mechanisms with minimal disruption to the legacy logic while still improving the protection of personal data, the transparency of processing, and the enforceability of policy constraints.

3.2.1 Target Architectural Principles

The target privacy architecture for the HMS should be based on the following principles:

1. **Purpose-driven access to data**

Data must be available only to the subjects and modules that require it for a legitimate operational function.

2. **Field-level minimization**

   Interfaces should expose the minimum set of attributes necessary for each role.

3. **Separation between identity, operational, and medical data**

   Different classes of information should not be propagated through the same views or logic flows without a clear necessity.

4. **Protected credential lifecycle**

   Authentication data must be isolated from business data and managed with secure storage and secure session practices.

5. **Accountability and traceability**

   Sensitive access and privileged actions should be auditable.

6. **Retention limitation and controlled deletion**

   Personal and operational data should not be retained indefinitely without policy.

3.2.2 Proposed Target Privacy Architecture

The re-engineered system can be conceptually divided into six privacy-relevant layers.

This layered representation also reflects the course emphasis on **defense in depth** and on the separation between architectural, implementation, and operational concerns. Privacy is therefore not confined to a single UI notice or a legal statement: it emerges from the joint behavior of access control, data handling, presentation logic, auditability, and retention governance.

3.2.2.1 Identity and Access Management Layer

This layer is responsible for authentication, authorization, session lifecycle, and access restrictions.

It should enforce:

- distinct roles for **patient**, **doctor**, and **administrator**;
- role-based access control (**RBAC**);

- object-level authorization checks on appointments, prescriptions, and personal records;
- session hardening (HttpOnly, Secure, SameSite, session regeneration after login);
- elimination of plaintext passwords in storage and presentation.

From a privacy perspective, this layer ensures that a user cannot access data simply because it exists, but only because the system verifies that the access is legitimate and role-consistent.

3.2.2.2 Secure Data Storage Layer

This layer governs how data is stored in the relational backend and how it is logically separated.

It should distinguish at least between:

- **identity/account data** (credentials, usernames, contacts);
- **operational scheduling data** (appointments, booking status, doctor assignment);
- **medical-support data** (prescriptions, allergies, disease notes);
- **communication data** (messages, feedback, contact requests);
- **audit data** (security-relevant logs and privileged actions).

The core privacy improvement here is not necessarily physical database sharding, but **logical separation of responsibilities and access rules**. For example:

- credential data should never be displayed in operational panels;
- medical notes should not be searchable or visible to roles that do not need them;
- feedback/contact data should not be mixed with patient clinical or scheduling data.

3.2.2.3 Privacy-Preserving Application Flow Layer

This layer regulates how data moves between forms, handlers, views, and server-side processing functions.

The current application frequently propagates full user records and medical details through multiple views and query flows. A privacy-aware redesign should instead apply:

- **purpose limitation by endpoint**;
- **minimal field projection in queries** (select only the columns needed);
- **restricted data binding in views**;
- **context-aware rendering**, where each page receives only the data required for its task.

Examples:

- a patient appointment history view should not expose administrative-only information;
- a doctor search result should not display irrelevant account secrets;
- an admin page should not surface patient passwords or unnecessary clinical details.

3.2.2.4 Presentation and Interface Minimization Layer

Privacy architecture also depends on how the UI renders information.

The target design should apply:

- no rendering of passwords or credential secrets;
- masking or omission of unnecessary identifiers;
- restricted exposure of PHI-like content in tables and search results;
- autocomplete disabled on sensitive forms where appropriate;
- explicit notices near data collection forms.

This layer is especially important because several Fortify findings point to privacy leakage through rendering behavior, not just through storage.

3.2.2.5 Audit and Accountability Layer

To satisfy the **Demonstrate** privacy strategy, the target architecture should include an accountability layer that records security- and privacy-relevant events.

Typical events to log include:

- administrative searches over patient or doctor data;

- appointment cancellation or modification actions;
- prescription creation or modification;
- privileged account operations;
- repeated failed authentication attempts.

The objective is not mass surveillance of ordinary users, but **traceability of sensitive processing actions**. This supports both privacy governance and incident investigation.

3.2.2.6 Retention and Deletion Governance Layer

The current system does not appear to implement explicit retention or deletion logic. A privacy-aware target architecture should define policies such as:

- retention limits for contact/feedback messages;
- rules for archival or deletion of obsolete appointment data;
- restricted retention of sensitive medical notes where educational scope permits;
- deletion or anonymization of unnecessary data after operational use ends.

This layer supports the principles of:

- storage limitation;
- data minimization over time;
- reduced long-term exposure of sensitive records.

3.2.3 Role-Centered Target View of the Architecture

The privacy architecture can also be described from the perspective of each role.

3.2.3.1 Patient Role

The patient should be able to:

- register and authenticate securely;
- view only their own appointments and personal data;
- submit booking or contact information with clear notice;

- avoid unnecessary exposure of internal hospital workflow data.

3.2.3.2 Doctor role

The doctor should be able to:

- access only the appointments and prescription-relevant data necessary for treatment workflow;
- avoid visibility into unrelated patient records;
- interact with minimized datasets rather than full account/admin views.

3.2.3.3 Administrator Role

The administrator should retain operational oversight but under stricter privacy constraints:

- broad visibility should be segmented by function;
- passwords and secrets must never be visible;
- clinical data should be visible only when necessary for legitimate administrative operations;
- privileged searches and modifications should be logged.

3.2.4 Architectural Improvements Mapped to Privacy Strategies

Architectural Control	Privacy Strategy	Expected Effect
RBAC + object-level authorization	Control / Enforce	Limits access to legitimate users and roles
Minimal field selection in queries and views	Minimize	Reduces overexposure of personal and medical data
Password hashing and secret isolation	Hide	Protects credentials from disclosure and misuse

Architectural Control	Privacy Strategy	Expected Effect
Separation of account, medical, and communication datasets	Separate	Prevents unnecessary data mixing across workflows
Audit logging of privileged actions	Demonstrate	Supports accountability and compliance evidence
Privacy notice and form transparency	Inform	Makes users aware of collection and usage purposes
Retention and deletion policy	Minimize / Enforce	Reduces long-term privacy exposure

3.2.4.1 Privacy Knowledge Base patterns selected for the HMS case study

Using the privacy knowledge base provided for the course, a subset of patterns was selected as the most relevant for the re-engineering of the Hospital Management System. The selection was not performed abstractly: each pattern was chosen because it helps address weaknesses that emerged concretely from Fortify, from the DAST-oriented analysis, and from the remediation work already performed on the PHP codebase.

The following table links the selected patterns to the corresponding HMS problems, privacy strategies, architectural placement, and expected benefit.

Privacy Issues, Strategy, and Selected Patterns

Observed HMS problem	Privacy strategy	Selected pattern from the knowledge base
Excessive visibility of patient, doctor, and appointment data in broad administrative/search views	**Control / Separate**	Selective access control
Over-propagation of user and medical data across unrelated views and workflows	**Separate**	User data confinement pattern
Need to represent and justify which personal data is collected, displayed, and retained	**Inform**	Personal Data Table
Lack of user-facing transparency regarding data collection, form purpose, and treatment logic	**Inform**	Privacy Policy Display
Weak or absent consent/awareness mechanisms in data-collection forms	**Control / Inform**	Obtaining Explicit Consent
Legacy insecure password handling and low user awareness regarding secure credential practices	**Inform**	Informed Secure Passwords
Need for a clearer role-based representation of what data is shown to whom	**Control**	Reasonable Level of Control

Observed HMS problem	Privacy strategy	Selected pattern from the knowledge base
Absence of an integrated user/admin view over privacy-relevant data handling choices	**Inform**	Privacy dashboard

MVC Positioning and Architectural Contribution

Selected pattern (Ref)	MVC placement	Expected contribution to the target architecture
Selective access control	Controller, Model	Restricts which role can see which record set and supports least-privilege visibility
User data confinement pattern	Controller, Model	Constrains data flows so that identity, operational, and medical information are not exposed outside their legitimate purpose
Personal Data Table	Controller, Model, View	Supports structured privacy documentation and explicit mapping of data categories, purpose, and visibility
Privacy Policy Display	Controller, Model, View	Improves transparency by making privacy information visible at the interface level

Selected pattern (Ref)	MVC placement	Expected contribution to the target architecture
Obtaining Explicit Consent	Controller, View	Introduces explicit consent checkpoints for privacy-relevant processing activities
Informed Secure Passwords	Controller, View	Links credential security to privacy awareness, encouraging secure password creation and safer authentication handling
Reasonable Level of Control	Controller, Model	Helps articulate how different users should gain access only to the data necessary for their function
Privacy dashboard	Controller, Model, View	Serves as a target-pattern proposal for improving accountability, visibility, and control over personal data processing

3.2.4.2 Why these patterns are appropriate for the HMS

The selected patterns are particularly suitable for the Hospital Management System because the project processes both **PII** and **PHI-like information** while exposing multiple role-dependent workflows. In this context, the central privacy problem is not only external compromise, but also **internal overexposure**: too much data is visible, too broadly, and often in contexts that do not strictly require it.

More specifically:

- **Selective access control** and **Reasonable Level of Control** respond to the need to reduce visibility in doctor/admin panels and to support object-level authorization;

- **User data confinement pattern** addresses the architectural problem that patient identity, contact, appointment, and quasi-clinical data currently circulate too broadly across views and handlers;
- **Personal Data Table** and **Privacy dashboard** help transform privacy from an implicit concern into an explicit, documentable design dimension;
- **Privacy Policy Display** and **Obtaining Explicit Consent** respond to the current lack of transparent privacy communication in the UI;
- **Informed Secure Passwords** bridges credential protection and privacy awareness, which is highly relevant given the historical plaintext password exposure identified in the original codebase.

3.2.4.3 Relationship between privacy patterns and the vulnerabilities identified in the HMS

The value of the knowledge-base patterns becomes clearer when they are mapped back to the concrete findings of the project:

Vulnerability / weakness observed in the HMS	**Related privacy pattern(s)**	**Rationale**
SQL Injection *in authentication and data retrieval flows*	Informed Secure Passwords, Selective access control, Reasonable Level of Control	Even though SQL Injection is primarily a security flaw, its remediation directly supports privacy by preventing unauthorized access to personal and medical data.
Excessive data exposure *in admin and search interfaces*	Selective access control, User data confinement pattern, Personal Data Table	These patterns help redesign visibility boundaries and minimize the fields exposed to each role.

Vulnerability / weakness observed in the HMS	**Related privacy pattern(s)**	**Rationale**
Privacy Violation / *Autocomplete issues*	Privacy Policy Display, Obtaining Explicit Consent, Privacy dashboard	These patterns strengthen user awareness and control over the handling of privacy-relevant inputs.
Weak password handling *in the original application*	Informed Secure Passwords	This pattern helps justify a privacy-aware approach to safer credential management.
Broad circulation *of patient data across views and operations*	User data confinement pattern, Reasonable Level of Control	These patterns reduce cross-context data propagation and support purpose limitation.
Lack of explicit , *structured privacy governance representation*	Personal Data Table, Privacy dashboard	These patterns improve accountability and make privacy architecture visible and demonstrable.

3.2.4.4 Methodological value for the case study

The use of the privacy knowledge base adds methodological strength to the case study because it allows the final privacy architecture to be presented not just as an ad hoc redesign, but as a pattern-informed re-engineering exercise. This is especially useful in an academic context: it demonstrates that the transition from vulnerability discovery to architectural improvement is guided by recognized privacy strategies and design knowledge rather than by isolated intuition alone.

3.2.5 Proposed Textual Target Architecture Model

The privacy-oriented re-engineered HMS can therefore be summarized as follows:

A role-segregated web architecture in which patient, doctor, and administrator workflows are isolated by authorization checks and by minimal data projection; credentials are stored securely and never rendered in plaintext; medical-support data is separated from administrative account management data; sensitive operations are logged; user-facing forms include transparency and retention-aware handling; and only the minimum required personal data is displayed, stored, or propagated across application components.

3.2.6 Relationship with Security Remediation

This privacy architecture is tightly connected to the security fixes already identified in Section 2.3. In this case study, privacy improvements do not replace security controls; rather, they depend on them. In particular:

- SQL Injection remediation protects personal and medical data from unauthorized extraction;
- XSS remediation protects browser-side confidentiality and session integrity;
- CSRF protection helps preserve the integrity of privacy-relevant operations;
- secure session management supports access confidentiality;
- data minimization reduces the blast radius of a compromise.

This means that the privacy architecture and the security fix plan should be presented as **mutually reinforcing dimensions** of the same re-engineering effort.

3.2.7 Expected Architectural Outcome

If the proposed privacy architecture were implemented, the Hospital Management System would evolve from a monolithic, data-exposing educational application into a more defensible platform characterized by:

- reduced unnecessary exposure of PII and PHI-like data;

- clearer separation of responsibilities between roles;
- stronger protection of authentication secrets;
- greater accountability for privileged actions;
- improved alignment with privacy-by-design and GDPR-oriented principles.

This target architecture therefore provides the conceptual endpoint of the case study: a secure and privacy-aware redesign of the original system based on evidence collected through reconnaissance, DAST, SAST, and privacy assessment.

Final Remarks

The case study now presents a coherent end-to-end analysis of the Hospital Management System, covering reconnaissance, vulnerability classification, remediation planning, privacy assessment, and privacy-oriented re-engineering. A particularly important result of the work is the convergence between:

- **static evidence** from Fortify SCA,
- **dynamic reasoning** based on DAST-oriented analysis,
- **source-code remediation examples** already applied to the project,
- **privacy-by-design measures** aligned with PKB-inspired strategies.

From a course-theory perspective, this final structure reflects the main pillars of Secure Software Engineering taught during the semester:

- **security concepts and risk reasoning**, through assets, threats, vulnerabilities, exploits, and CIA impact;
- **attack modeling**, through reconnaissance and kill-chain-oriented reasoning;
- **secure coding and verification**, through SAST, DAST, and manual validation of SQLi/XSS/CSRF mitigations;
- **access control theory**, through the emphasis on role separation, need-to-know, implicit deny, and accountability;
- **privacy-oriented software development**, through POSD, Privacy by Design principles, privacy strategies, and knowledge-base-driven pattern selection.

From an educational and engineering perspective, the case study demonstrates that security and privacy must be treated as mutually reinforcing properties of the same software lifecycle. The most critical technical weaknesses identified in the original application—especially SQL Injection, weak password handling, unsafe rendering, and privacy overexposure—show how a relatively simple web application can become highly risky when sensitive healthcare-related data is processed without adequate protections.

At the same time, the remediation and target architecture sections show that these weaknesses can be systematically reduced through:

- secure coding practices,
- constrained data exposure,
- stronger access control,
- accountability mechanisms,
- and architectural privacy controls.

The remaining open issues—particularly residual XSS review across large panels, deeper authorization validation against IDOR-like cases, and additional browser/deployment hardening—should therefore be interpreted not as a failure of the case study, but as the natural next step in the progressive secure re-engineering of the application.